

CS-250: Lab 07

Implementation of Stacks in Different Problems

Muhammad Mujtaba

CMD ID: 540040

mmujtaba.bese25seecs@seecs.edu.pk

Class: BESE 16B

Batch: 2k25

Stack DS Used:

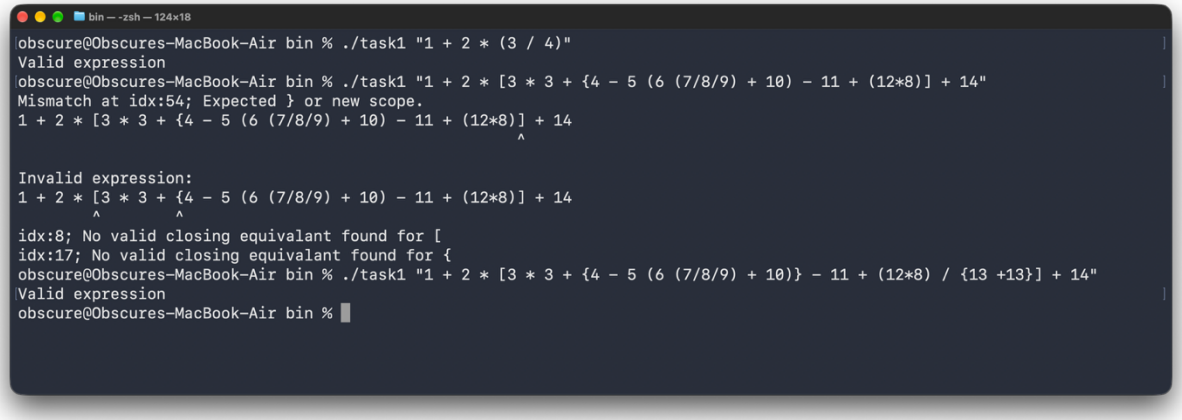
See stack.hpp.

GitHub Repo Link: https://github.com/mmujtaba25/CS-250

Zip File for only Lab 06: https://github.com/mmujtaba25/CS-250/releases/tag/lab-07

Task 1:

See task1.cpp.



Task Driver:

```
int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        std::println("Pass in the expression as the first argument when running");
        return -1;
    }

    std::string expression = argv[1];
    Stack<std::pair<size_t, char>> brackets_pair;

    auto handleClosingBracket = [&brackets_pair, &expression](size_t index, char openingBracket) -> void { //
        if (!brackets_pair.isEmpty() && brackets_pair.peek()->second == openingBracket)
            brackets_pair.pop();
        else
        {
            std::println("Mismatch at idx:{}; Expected {} or new scope.", index, equivalantBracket(brackets_pair.peek()-
>second).value());
            highlightStackElement(expression, {index});
        }
    };

    for (size_t i = 0; i < expression.size(); i++)
    {
        const char &c = expression[i];

        switch (c)
        {
        case '(':
        case '{':
        case '[':
            brackets_pair.push(std::make_pair(i, c));
            break;
```

```
        case ')':
        case '}':
        case ']':
            handleClosingBracket(i, equivalantBracket(c).value_or(' '));
            break;

        // skip any other character
        default:
            continue;
        }
    }

    if (brackets_pair.isEmpty())
        std::println("Valid expression");
    else
    {
        std::println("\nInvalid expression:");
        std::vector<size_t> indexes;

        for (const auto &[index, bracket] : brackets_pair)
            indexes.push_back(index);

        highlightStackElement(expression, indexes);

        for (int i = indexes.size(); i > 0; i--)
        {
            size_t index = indexes[i - 1];
            std::println("idx:{}; No valid closing equivalant found for {}", index, expression[index]);
        }
    }

    return 0;
}
```

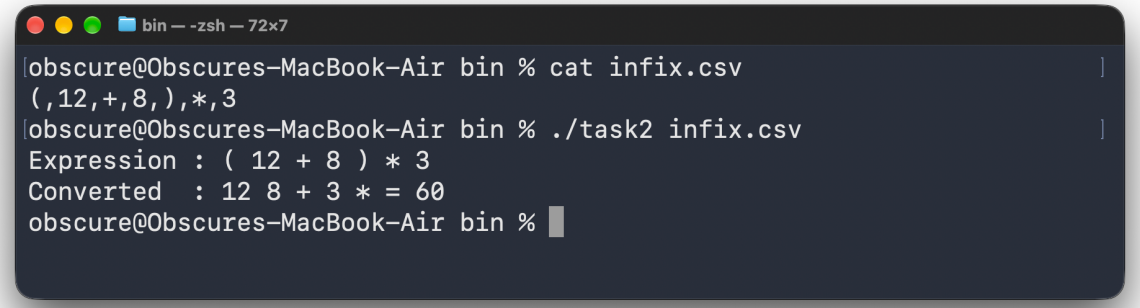
Task 2:

See task2.cpp.

Part A: task2.cpp#L51

Part B: task2.cpp#L79

Part C: task2.cpp#L118



Task Driver:

```cpp
int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        std::println("Pass in the filename as the first argument when running");
        return -1;
    }

    std::string infixFile = argv[1];

    std::vector<std::string> infix = readCSV(infixFile);

    std::print("Expression : ");
    for (const std::string &token : infix)
    {
        std::print("{} ", token);
    }
    std::println();

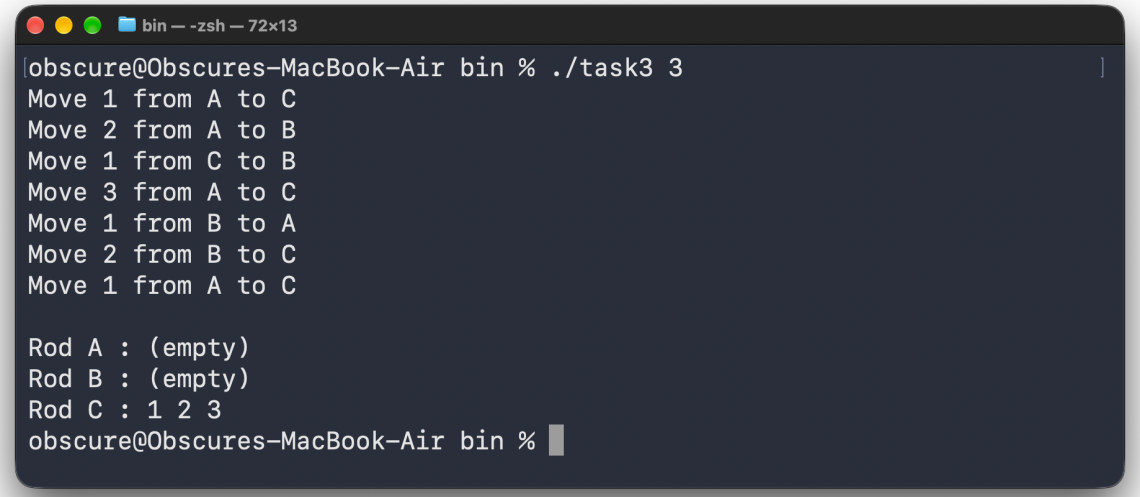
    std::vector<std::string> tokens = infixToPostfix(infix);

    std::print("Converted  : ");
    for (const std::string &token : tokens)
    {
        std::print("{} ", token);
    }
    std::print("= {}\n", evaluatePostfix(tokens));

    return 0;
}
```

Task 3:

See task3.cpp.



Task Driver:

```cpp
int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        std::println("Pass in the number of disks as the first argument");
        return 1;
    }

    auto result = isNumber(argv[1]);

    if (!result || *result < 1 || *result > 63 ||
        *result != static_cast<int>(*result))
    {
        std::println("Enter a positive integer between 1 and 63");
        return 1;
    }

    int numDisks = static_cast<int>(*result);

    Rod rodA{.name = 'A'};
    Rod rodB{.name = 'B'};
    Rod rodC{.name = 'C'};

    // largest disk at the bottom
    for (uint8_t disk = numDisks; disk >= 1; --disk)
        rodA.disks.push(disk);

    towerOfHanoi(numDisks, rodA, rodB, rodC);

    std::println();
    printRod(rodA, rodA.name);
    printRod(rodB, rodB.name);
    printRod(rodC, rodC.name);

    return 0;
}
```